

Guide de preparation - Questions de soutenance

Projet Machine Learning - ISE | Module Segmentation Client | 2026

Ce document couvre les questions les plus probables selon le profil de la professeure : 50% business/storytelling, 50% technique. Adaptez les reponses avec vos propres mots.

1. CONTEXTE BUSINESS ET STORYTELLING

Q1. Pourquoi segmenter les clients Olist ? Quel probleme business concret resolvez-vous ?

Reponse : Sans segmentation, toutes les campagnes marketing sont identiques pour tout le monde. On depense autant pour un client actif que pour un perdu depuis 1 an. Notre segmentation permet d'allouer le budget la ou le ROI est le plus eleve : relancer un dormant premium (-20%) est justifie a 160 BRL de panier, pas un dormant budget a 72 BRL.

Q2. Decrivez vos 4 segments en termes business, sans jargon technique.

Reponse : C0 - Acheteurs Budget (28%) : recents, petit panier (~64 BRL). Action : promotions frequentes. C1 - Dormants Budget (25%) : inactifs ~10 mois, faible valeur. Action : email simple, ROI faible. C2 - Acheteurs Premium (24%) : recents, gros panier (~180 BRL). Action : programme VIP, nurturing J+30. C3 - Dormants Premium (23%) : inactifs ~12 mois, fort potentiel (~160 BRL). Action : offre -20% urgente.

Q3. Quel est votre segment le plus prioritaire et pourquoi ?

Reponse : C2 - Acheteurs Premium. Clients recents encore engages, panier le plus eleve. Sans action dans les 60 jours, ils basculent en C3. ROI le plus immediat.

Q4. Qu'est-ce que le CLV Proxy ?

Reponse : Approximation de la valeur totale d'un client : $\text{Monetary} * (1 + \text{Frequency_flag}) / (1 + \text{Recency_normalise})$. Permet de comparer les segments sans donnees futures.

Q5. Comment vos segments aident-ils concrètement les equipes marketing ?

Reponse : Chaque segment a une fiche dans le dashboard : KPIs, categories de produits, ciblage pub (age, genre, revenus, Facebook/Google) et 5 actions generees par IA Gemini. L'equipe configure une campagne directement sans comprendre le ML.

2. DONNEES ET EXPLORATION (EDA)

Q6. Pourquoi customer_unique_id et pas customer_id ?

Reponse : Un meme client peut avoir plusieurs customer_id selon l'adresse utilisee. customer_unique_id est le seul identifiant representant vraiment une personne unique.

Q7. 97% de clients n'ont commande qu'une fois. Comment avez-vous gere ca ?

Reponse : Frequency transformee en flag binaire (0 = un achat, 1 = au moins 2). Une variable quasi-binaire ne discrimine pas les clusters. La segmentation repose sur Recency et Monetary.

Q8. Pourquoi la mediane plutot que la moyenne pour profiler les clusters ?

Reponse : Les distributions sont asymetriques. Un client a 10 000 BRL fait exploser la moyenne. La mediane represente le client typique du segment.

Q9. Comment avez-vous traite les outliers ?

Reponse : Clipping aux percentiles 5%-95%, jamais de suppression. Les outliers sont de vrais clients. Le clipping les ramene a des bornes raisonnables sans les eliminer.

Q10. Qu'est-ce que le region_freight_score ?

Reponse : Score ordinal du cout de livraison par region. Les etats du Nord paient 2 a 3 fois plus que Sao Paulo. Permet de capturer l'effet geographique sans encoder 27 etats.

3. FEATURES, MODELES ET ORDRE DE LA DEMARCHE

ATTENTION : Benchmark (notebook 03) : 9 features, 6 algorithmes. Production (generate_artifacts.py) : 3 features RFM + UMAP + KMeans. Distinguez ces deux etapes clairement.

ORDRE A RETENIR : ETAPE 1 : Baseline 16 features -> silhouette 0.21. ETAPE 2 : Benchmark 6 algo sur 9 features -> meilleur eligible = KMeans (0.22). ETAPE 3 : KMeans + UMAP + 3 features RFM -> silhouette 0.45. DBSCAN et HDBSCAN exclus a priori (transductifs).

Q11. Combien de features utilise votre modele final ? Et pour le benchmark ?

Reponse : Benchmark (notebook 03) : 9 features sur 6 algorithmes (KMeans, BisectingKMeans, CAH, GMM, DBSCAN, HDBSCAN). Silhouette ~0.22 pour les eligibles. Production : 3 features RFM + UMAP -> silhouette 0.45. Le double.

Q12. Pourquoi 3 features en production au lieu des 9 du benchmark ?

Reponse : RFM (Recency, Frequency, Monetary) est le standard de la segmentation client en e-commerce. Ces 3 axes suffisent a definir le comportement d'achat. Les 6 autres features ne sont pas supprimees : elles servent a enrichir et interpreter les profils apres le clustering, pas a definir les groupes.

Q13. Pourquoi avoir specifiquement pense a UMAP plutot qu'une autre technique ?

Reponse : On avait besoin d'une reduction dimensionnelle non-lineaire et compatible production. PCA est lineaire donc ne capture pas les relations complexes entre comportements clients. t-SNE est non-lineaire mais transductif (pas de transform() sur un nouveau client) et trop lent sur 75 000 clients. UMAP combine les 3 qualites requises : non-lineaire, possede transform(), et rapide. C'est aussi une combinaison UMAP + KMeans bien documentee dans la litterature de clustering.

Q14. Comment savez-vous que c'est UMAP et pas la reduction de 9 a 3 features qui explique le gain ?

Reponse : On ne peut pas l'isoler avec certitude : ablation incomplete. On a teste KMeans + 9 features sans UMAP (0.22) et KMeans + 3 features + UMAP (0.45), mais pas les deux combinaisons intermediaires. Hypothese : UMAP est le facteur dominant, car passer de 16 a 9 features sans UMAP n'avait rien change. C'est une limite methodologique qu'on assume.

LIMITE A ASSUMER : Si la prof insiste : 'Nous n'avons pas fait l'ablation complete, c'est une limite. Une etude rigoureuse aurait teste les 4 combinaisons separement.' Reconnaître une limite avec recul est valorise.

Q15. Pourquoi 6 algorithmes ? Lesquels ?

Reponse : 4 familles de clustering testees sur les memes 9 features : Centroides (KMeans, BisectingKMeans), Hierarchique (CAH), Distribution (GMM), Densite (DBSCAN, HDBSCAN). Tester plusieurs familles garantit qu'on ne rate pas une structure dans les donnees.

Q16. Pourquoi DBSCAN et HDBSCAN ont-ils ete exclus ?

Reponse : Exclusion a priori : ils sont transductifs, pas de predict(). Impossible de classifier un nouveau client sans re-entraîner sur toutes les donnees. Leurs scores ont ete mesures pour comparaison informative uniquement.

Q17. Pourquoi la reduction 16->9 features appliquee a tous les modeles ?

Reponse : Pour un benchmark equitable. Donner des configurations differentes a chaque algorithme comparerait des pipelines, pas les algorithmes eux-memes.

Q18. Pourquoi UMAP seulement sur KMeans et pas sur tous les modeles ?

Reponse : UMAP est une optimisation appliquee apres le benchmark sur le meilleur eligible. L'appliquer a tous aurait compare des pipelines differents. De plus, UMAP ne resout pas les problemes fondamentaux de CAH ($O(n^2)$) ni de DBSCAN/HDBSCAN (transductifs).

Q19. Pourquoi une log-transformation sur Recency et Monetary ?

Reponse : K-Means utilise des distances euclidiennes. Avec skewness > 3, un client a 5000 BRL domine tous les calculs. $\log_{10}(x)$ compresse les queues. Skewness passe de >3 a ~0.5.

Q20. Pourquoi StandardScaler ?

Reponse : Donne un poids egal a toutes les features dans K-Means (moyenne=0, ecart-type=1). Apres clipping des outliers, c'est suffisant et standard dans la litterature.

Q21. Pourquoi le scaler dans le notebook 03 et pas dans le 02 ?

Reponse : Pour eviter le data leakage. Integre dans le pipeline sklearn sauvegarde, il s'applique via transform() sur un nouveau client sans jamais refaire fit().

4. CHOIX ALGORITHMIQUES ET METRIQUES

Q22. Pourquoi le Silhouette Score comme metrique principale ?

Reponse : Mesure simultanement la cohesion intra-cluster et la separation inter-clusters. Varie de -1 a +1. Score > 0.20 acceptable sur donnees comportementales. Notre 0.449 est excellent. Complete par Davies-Bouldin et Calinski-Harabasz dans un score composite.

Q23. Pourquoi avoir ecarte la CAH ?

Reponse : Silhouette de 0.18 contre 0.22 pour KMeans. Et complexite $O(n^2)$ en memoire : ~45 GB de RAM sur 75 000 clients. Impraticable en production.

Q24. Pourquoi k=4 ?

Reponse : Methode du coude et maximisation du Silhouette Score convergent vers k=4. Au-dela : gain marginal, interpretabilite en baisse. En deca : distinctions importantes perdues (Premium actifs/inactifs fusionnes).

Q25. Comment garantissez-vous la stabilite dans le temps ?

Reponse : Simulation temporelle (notebook 04) sur des sous-periodes. Re-entrainement recommande tous les 3-6 mois.

5. RESULTATS ET CRITIQUES

Q26. 0.449 de Silhouette : bon ou mauvais ?

Reponse : Tres bon. Score > 0.20 = acceptable, > 0.40 = bien structure. Notre baseline a 16 features donnait 0.21. UMAP + 3 features RFM : +100%.

Q27. Un simple croisement Recency/Monetary aurait suffi. Pourquoi du ML ?

Reponse : Un croisement manuel impose des seuils arbitraires. K-Means + UMAP trouve les frontieres optimales automatiquement, de facon rigoureuse et reproductible. Et le modele peut classifier un nouveau client en temps reel.

Q28. Pourquoi pas un modele supervise ?

Reponse : La segmentation est non supervisee : on cherche des groupes latents, pas a predire une variable cible. Le review_score est une feature d'enrichissement, pas une cible.

Q29. Si Olist vous donne de nouvelles donnees dans 6 mois ?

Reponse : Mettre a jour la date de reference RFM, puis : predict() si derive faible, ou re-entrainer via generate_artifacts.py si derive significative. Re-evaluation recommandee tous les 3-6 mois.

6. DASHBOARD ET DEPLOIEMENT

Q30. Pourquoi Streamlit ?

Reponse : Python pur, sans frontend. Meilleur compromis rapidite/qualite pour une equipe data. Power BI ne permet pas d'integrer des modeles ML custom ni Gemini.

Q31. Comment fonctionne la page de prediction ?

Reponse : Entree client -> StandardScaler -> UMAP.transform() -> KMeans.predict(). KMeans est inductif : on classe un nouveau client sans re-entrainer.

Q32. Pourquoi Gemini dans le dashboard ?

Reponse : Les segments donnent des chiffres. Les equipes marketing ont besoin d'actions en langage naturel. Gemini genere 5 actions personnalisees par segment.

Q33. Avez-vous fait du data leakage ?

Reponse : Non. StandardScaler dans le pipeline sklearn, entraine une seule fois. Prediction via transform() et predict(), jamais fit() sur nouvelles donnees.

Q34. Comment est deploye le projet ?

Reponse : Docker (python:3.10-slim) + Google Cloud Run (serverless). Cles API en variables d'environnement, jamais dans le code.

Q35. Vos profils de clusters mentionnent l'age, le genre... Ces variables ne sont pas dans la base. Comment c'est possible ?

Reponse : Ce sont nos propres deductions, pas des donnees directes. Olist ne contient aucune information demographique. Pour construire ces profils, on a utilise deux sources : d'abord les 3 features RFM du modele, qui donnent le comportement d'achat brut ; ensuite les 6 features ecartees du modele (satisfaction, retards de livraison, panier moyen, duree de vie client...) qui n'ont pas servi au clustering mais ont ete tres utiles pour affiner et enrichir la description de chaque segment. A partir de tout ca, on interprete : un cluster qui achete surtout des jouets avec un panier familial eleve sugere un profil de parents ; un cluster concentre sur l'electronique avec des commandes tardives sugere un jeune actif urbain. Ce sont des hypotheses marketing formulees manuellement. Gemini ne fait pas ces descriptions : son role est uniquement de generer les 5 recommandations d'actions marketing par segment.

7. MLOPS ET INFRASTRUCTURE

POURQUOI MLOPS ? : Un modele ML n'a de valeur que s'il est deploye, monitore et maintenable. MLOps = pratiques qui font le pont entre le notebook et la production. La prof peut questionner chaque composant de la stack.

Q36. Qu'est-ce que le MLOps et pourquoi l'avez-vous integre dans ce projet ?

Reponse : MLOps (Machine Learning Operations) = ensemble des pratiques pour deployer, monitorer et maintenir des modeles ML en production de facon fiable et reproductible. Sans MLOps, le modele reste un prototype dans un notebook. Avec MLOps, il devient un service interrogeable, versionne et maintenable.

Q37. A quoi sert MLflow dans votre projet ?

Reponse : MLflow est notre systeme de tracking d'experiences. Pour chaque entraînement, il enregistre : les hyperparametres (k, n_neighbors, n_components), les metriques (Silhouette Score, Davies-Bouldin), et les artefacts (modele pickle, graphiques). 350 runs loggues au total. Cela permet de comparer objectivement les configurations et de reproduire n'importe quel resultat passe.

Q38. Comment fonctionne votre pipeline CI/CD avec GitHub Actions ?

Reponse : A chaque push sur main : (1) GitHub Actions declenche le workflow. (2) Installation des dependances (requirements.txt). (3) Execution des tests unitaires Pytest. (4) Si les tests passent, build de l'image Docker. (5) Push sur Google Artifact Registry. (6) Deploiement automatique sur Google Cloud Run. Ce pipeline garantit que seul du code teste arrive en production.

Q39. Pourquoi Docker ? Qu'apporte la conteneurisation ?

Reponse : Docker garantit que le code tourne de facon identique partout : sur le MacBook de dev et sur le serveur cloud. Toutes les dependances sont packagees dans l'image, donc plus de probleme de version ou de configuration. C'est la condition de base pour deployer sur Google Cloud Run.

Q40. Pourquoi Google Cloud Run plutot qu'une VM ou Kubernetes ?

Reponse : Cloud Run démarre automatiquement quand quelqu'un accede au dashboard et s'eteint quand personne ne l'utilise. Cout = 0 sans trafic. Pas de serveur a configurer ni a maintenir. Pour un projet de cette taille, c'est le choix le plus simple et le plus economique.

Q41. Comment gerez-vous le versioning des modeles ?

Reponse : Chaque modele entraine est enregistre dans MLflow avec un numero de version. Si le nouveau modele est moins bon que le precedent, on peut revenir a l'ancienne version en une commande. Les images Docker sont aussi versionnees via Git, donc le code et le modele sont toujours synchronises.

Q42. Qu'est-ce que le data drift et comment le detecteriez-vous ?

Reponse : Le data drift = changement de distribution des donnees d'entree au fil du temps. Exemple : une promotion Olist attire soudain beaucoup de nouveaux acheteurs uniques, la distribution de Recency change, les clusters ne correspondent plus aux profils initiaux. Detection : monitoring mensuel (moyenne, ecart-type, KS-test) des 3 features RFM. Si p-value KS-test < 0.05, on declenche un re-entraînement.

Q43. Avez-vous des tests unitaires dans ce projet ?

Reponse : Oui. On a deux fichiers de tests dans le dossier tests/ : un pour le feature engineering, un pour les fonctions du modele. Ils verifient que le code produit les bons resultats sur des donnees controlees, par exemple que les transformations RFM retournent les bonnes colonnes ou que la prediction retourne bien un numero de cluster valide. Ces tests tournent automatiquement a chaque push via GitHub Actions.

Q44. Comment garanteez-vous la reproductibilite des resultats ?

Reponse : 3 mecanismes : (1) random_state=42 sur KMeans, UMAP et tous les splits. (2) requirements.txt avec versions figees (scikit-learn==1.3.0, umap-learn==0.5.3...). (3) MLflow logue les hyperparametres exacts de chaque run. Avec ces 3 elements, n'importe qui peut reproduire exactement notre Silhouette Score de 0.4486 en re-executant le notebook 03.

Q45. Pourquoi PostgreSQL Neon plutot qu'un simple fichier CSV ?

Reponse : 4 raisons : (1) Scalabilite : PostgreSQL gere des milliards de lignes, un CSV est limite. (2) SQL natif : les jointures multi-tables (orders, items, payments, reviews) sont optimisees. (3) Concurrence : plusieurs services peuvent lire simultanement. (4) Production-ready : les APIs se connectent via SQLAlchemy exactement comme en environnement professionnel. Neon = PostgreSQL serverless, scale a zero quand inactif.

Q46. Si le modele derive en production, comment reagissez-vous ?

Reponse : Detection : monitoring mensuel des distributions RFM (KS-test), suivi du Silhouette Score sur nouvelles donnees (alerte si < 0.35), verification de la repartition des clusters (signal fort si un cluster passe de 8% a 30%). Reaction : re-entrainement sur les 12 derniers mois, validation du nouveau modele (Silhouette + interpretabilite), deploiement via CI/CD avec rollback automatique si regression.

Q47. Quels workflows GitHub Actions avez-vous crees ? Qu'est-ce que CI et CD concretement dans votre projet ?

Reponse : Nous avons 4 workflows dans .github/workflows/ :

CI (ci.yml) : se declenche a chaque push sur n'importe quelle branche. Il installe les dependances, verifie le formatage du code avec Black, puis execute les tests unitaires avec Pytest. Si un test echoue, le code ne peut pas etre merge.

CD (cd.yml) : se declenche uniquement quand le code arrive sur la branche main. Il construit l'image Docker, la pousse sur Google Artifact Registry, puis deploie automatiquement la nouvelle version sur Google Cloud Run.

Ingestion (ingest.yml) : se declenche quand un nouveau fichier CSV arrive dans data/incoming/. Il ingere les donnees dans PostgreSQL Neon, puis relance le re-entrainement si de nouvelles donnees ont ete ajoutees.

Re-entrainement trimestriel (retrain.yml) : planifie automatiquement le 1er janvier, avril, juillet et octobre. Il re-entraine le modele et ne le remplace en production que si le nouveau Silhouette Score est meilleur que l'ancien.

Q48. Comment gerez-vous le re-entrainement du modele ?

Reponse : On re-entraine tous les 3 a 6 mois, ou plus tot si on detecte une derive des donnees. Le re-entrainement se fait en re-executant le script generate_artifacts.py sur les donnees les plus recentes. Le nouveau modele est compare a l'ancien via le Silhouette Score : s'il est meilleur, il remplace l'ancien en production via le pipeline CI/CD. Sinon, on garde l'ancien. Tout est versionne dans MLflow, donc on ne perd jamais un modele precedent.

A RETENIR POUR LA SOUTENANCE : MLOps n'est pas une option : c'est ce qui distingue un projet ML académique d'un système en production. Chaque outil de la stack a une justification précise. Si la prof demande 'pourquoi pas X ?', répondre : comparaison faite, X exclus pour [raison concrète].

8. BASE DE DONNÉES ET INGESTION

SCHEMA DE LA BASE : 8 tables chargées depuis les CSV Olist : olist_orders, olist_customers, olist_order_items, olist_products, product_category_name_translation, olist_sellers, olist_order_payments, olist_order_reviews. Toutes les jointures se font en SQL, pas en Pandas.

Q49. Comment avez-vous chargé les données CSV dans PostgreSQL ?

Reponse : Via la fonction `ingest_csv_to_postgres()` dans `src/data_loader.py`. Elle parcourt tous les fichiers CSV du dossier `data/`, lit chaque fichier avec Pandas, puis l'uploade dans PostgreSQL avec `df.to_sql()` en mode 'replace' (écrase la table si elle existe déjà). Le nom de la table est dérivé du nom du fichier CSV. En pratique, les données viennent aussi directement de l'API Kaggle via `download_kaggle_dataset()` qui télécharge et dézippe le dataset automatiquement.

Q50. Comment votre code Python se connecte-t-il à la base de données ?

Reponse : Via SQLAlchemy et la fonction `get_db_engine()` dans `data_loader.py`. Elle gère deux modes : en production (Neon), elle lit la variable d'environnement `DATABASE_URL` et crée un moteur avec SSL obligatoire. En développement local, elle lit les variables `POSTGRES_USER`, `PASSWORD`, `HOST`, `PORT`, `DB` depuis le fichier `.env` et se connecte au conteneur Docker. Le reste du code ne fait pas la différence : il reçoit juste un engine SQLAlchemy.

Q51. Comment gérez-vous les jointures entre les 8 tables ?

Reponse : Toutes les jointures sont faites directement en SQL dans `data_loader.py`, pas en Pandas. La fonction `get_merged_dataframe()` exécute une requête SQL qui joint les 7 tables principales avec des JOIN et LEFT JOIN, filtrée sur les commandes livrées (`order_status = 'delivered'`). Pourquoi en SQL ? PostgreSQL optimise les jointures nativement. Faire la même chose en Pandas avec des `merge()` successifs sur 100 000 lignes serait beaucoup plus lent et consommerait plus de mémoire.

Q52. Comment fonctionne l'environnement local vs la production pour la base ?

Reponse : En local : on lance un conteneur PostgreSQL 15 avec `docker-compose` (port 5444 sur la machine). Les credentials sont dans le fichier `.env`. En production (Cloud Run) : on se connecte à Neon via `DATABASE_URL`, une variable d'environnement injectée par GitHub Actions au déploiement. Le code est identique dans les deux cas : seule la variable d'environnement change. Les credentials ne sont jamais écrits dans le code.

Q53. Pourquoi avoir fait les agrégations en SQL plutôt que de tout charger en Pandas ?

Reponse : Performance et scalabilité. La fonction `get_customer_aggregation()` fait directement en SQL : COUNT des commandes, SUM des paiements, AVG des notes, GROUP BY `customer_unique_id`. PostgreSQL traite ça en une seule passe sur les données indexées. Charger toutes les tables brutes en Pandas puis faire les `merge()` et `groupby()` en Python aurait chargé plusieurs centaines de Mo en RAM inutilement. En SQL, seul le résultat agrégé (~96 000 lignes) remonte en Python.